Python for ML

02/12/2026

Why Python for ML?

Why Python for ML

- Simple, readable syntax

```
for i in range(5):
    print(i)
```

- Huge ecosystem
- Research-to-production pipeline
- Community support
- Strong GPU integration

What Does “Python for ML” Actually Mean?

- Loading data
- Cleaning data
- Visualizing data
- Training models
- Evaluating models
- Deploying models

Loading Data

```
import pandas as pd

url = "MachineLearning/Python4ML.csv"
df = pd.read_csv(url)

print(df.shape)
print(df.head(3))
print(df.dtypes)
```

Cleaning Data

```
import pandas as pd

# Basic missing-value report
missing = df.isna().mean().sort_values(ascending=False)
print(missing.head(6))

# Drop a column with too many missing values
df = df.drop(columns=["deck"])

# Fill missing age with median
df["age"] = df["age"].fillna(df["age"].median())
```

Visualizing Data

- **Matplotlib**
  - Basic plotting library in Python
  - Full control over plots
  - Supports line, bar, scatter, histogram, etc.
  - Foundation of many other visualization libraries

- **Seaborn**
  - Built on top of Matplotlib
  - More attractive default styles
  - Easy statistical visualizations

Training Models

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
```

Evaluating Models

```
from sklearn.metrics import accuracy_score

# Predict on test set
y_pred = model.predict(X_test)

# Accuracy
acc = accuracy_score(y_test, y_pred)
print("Test accuracy:", acc)
```

Development Environment

Interactive Environments

- **Jupyter Notebook**
  - A web application that allows you to create and share documents
  - Two main cell types:
    - Code cells (run Python)
    - Markdown cells (notes, equations, explanations)

A gallery of interesting Jupyter Notebooks
Data Science iPython Notebooks
iPython Notebooks for the textbook: Python for Data Analysis

Interactive Environments

- **Google Colab**
  - A hosted Jupyter Notebook in the browser
  - No installation required
  - Easy sharing like Google Docs
  - Often includes free compute (CPU/GPU/TPU)

Open Source Python Libraries

NumPy (Numerical Python)

- Core library for numerical computing
- Core data structure: ndarray (n-dimensional array)
- Efficient computation on large arrays
- Foundation of most ML libraries

Creating NumPy Arrays

```
import numpy as np

a = np.array([1, 2, 3])
b = np.zeros((2, 3))
c = np.random.randn(3, 3)

print(a)
print(b)
print(c)
```

```
[1 2 3]
[[0. 0. 0.]
 [0. 0. 0.]]
[[-0.00694085 -0.80273486 -1.42502575]
 [ 0.53442894  0.5571894   0.35735431]
 [ 2.15217546 -1.51618955 -0.41777183]]
```

Array Shape and Dimensions

```
x = np.random.randn(5, 3)

print(x.shape)
print(x.size)
```

```
(5, 3)
15
```

Indexing and Slicing

```
x = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print(x[0, 1])
print(x[:, 0])
print(x[1:, 1:])
```

```
20
[10 40 70]
[[50 60]
 [80 90]]
```

Vectorized Operations

Vectorization means performing operations on **entire arrays at once**, instead of using loops.

```
x = np.array([1, 2, 3, 4])

print(x + 10)
print(x * 2)
print(x ** 2)
```

```
[11 12 13 14]
[2 4 6 8]
[ 1  4  9 16]
```

Broadcasting

Broadcasting allows NumPy to **perform operations on arrays with different shapes.**

```
X = np.array([[1, 2, 3],
              [4, 5, 6]])

mean = X.mean(axis=0)

print('column-wise mean:', mean)

X_centered = X - mean

print('X centered:', X_centered)
```

```
column-wise mean: [2.5 3.5 4.5]
X centered: [[-1.5 -1.5 -1.5]
 [ 1.5  1.5  1.5]]
```

Linear Algebra in NumPy

```
X = np.random.randn(100, 5)
w = np.random.randn(5, 1)

y = X @ w

print(y.shape)
```

```
(100, 1)
```

References & Resources

- NumPy Quickstart
- NumPy for Beginners
- NumPy Broadcasting Explained
- Linear Algebra with NumPy

Pandas (Python Data Analysis Library)

- Built on top of NumPy
- Provides *DataFrame* and *Series*
- Designed for tabular data
    - Tabular data is data arranged in rows and columns
- Essential for data cleaning & preparation

Creating and Loading Data

```
import pandas as pd

df = pd.read_csv("students.csv")

df.head(5)
```

	student_id	age	gender	math	science	english	total	average	passed
0	1	20	Female	70	84	82	236	78.67	1
1	2	21	Female	65	86	50	201	67.00	1
2	3	19	Female	94	96	76	266	88.67	1
3	4	21	Male	67	63	62	192	64.00	1
4	5	21	Female	96	52	90	238	79.33	1

Creating and Loading Data

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype  
---  ------      --------------  -----  
 0   student_id  50 non-null     int64  
 1   age         50 non-null     int64  
 2   gender      50 non-null     object 
 3   math        50 non-null     int64  
 4   science     50 non-null     int64  
 5   english     50 non-null     int64  
 6   total       50 non-null     int64  
 7   average     50 non-null     float64
 8   passed      50 non-null     int64  
dtypes: float64(1), int64(7), object(1)
memory usage: 3.6+ KB
```

Selecting Columns

```
df["age"]              # select one column
df[["age", "gender"]]   # select multiple columns
```

	age	gender
0	20	Female
1	21	Female
2	19	Female
3	21	Male
4	21	Female
5	18	Male
6	19	Female
7	19	Female
8	19	Female
9	21	Male
10	20	Female
11	19	Male
12	22	Female
13	21	Male
14	18	Female

Filtering Data

```
df[df["age"] > 18]
```

	student_id	age	gender	math	science	english	total	average	passed
0	1	20	Female	70	84	82	236	78.67	1
1	2	21	Female	65	86	50	201	67.00	1
2	3	19	Female	94	96	76	266	88.67	1
3	4	21	Male	67	63	62	192	64.00	1
4	5	21	Female	96	52	90	238	79.33	1
6	7	19	Female	75	54	88	217	72.33	1
7	8	19	Female	74	75	55	204	68.00	1
8	9	19	Female	94	63	57	214	71.33	1
9	10	21	Male	90	88	76	254	84.67	1
10	11	20	Female	78	76	58	212	70.67	1
11	12	19	Male	64	58	86	208	69.33	1
12	13	22	Female	94	64	82	240	80.00	1
13	14	21	Male	50	64	91	205	68.33	1
15	16	20	Male	56	91	73	220	73.33	1

Filtering Data

```
df[(df["age"] > 18) & (df["total"] > 250)]
```

	student_id	age	gender	math	science	english	total	average	passed
2	3	19	Female	94	96	76	266	88.67	1
9	10	21	Male	90	88	76	254	84.67	1
19	20	20	Female	93	98	73	264	88.00	1
29	30	19	Female	91	78	98	267	89.00	1
37	38	19	Female	83	91	81	255	85.00	1
38	39	20	Female	82	94	82	258	86.00	1
42	43	21	Male	86	93	93	272	90.67	1
43	44	19	Female	84	93	75	252	84.00	1

Creating New Features

```
df["age_group"] = df["age"] // 10
df.head(5)
```

student_id	age	gender	math	science	english	total	average	passed	age_group
1	20	Female	70	84	82	236	78.67	1	2
2	21	Female	65	86	50	201	67.00	1	2
3	19	Female	94	96	76	266	88.67	1	1
4	21	Male	67	63	62	192	64.00	1	2
5	21	Female	96	52	90	238	79.33	1	2

References & Resources

- Official Documentation
- 10 Minutes Tutorial
- Pandas Overview

SciPy (Scientific Computing)

- Built on top of NumPy
- Provides advanced scientific functions
    - Optimization
    - Linear Algebra
    - Statistics
    - Signal Processing

scipy.optimize

```
from scipy.optimize import minimize
import numpy as np

def f(x):
    return x**2 + 3*x + 2

result = minimize(f, x0=0)
print(result.x)
```

```
[-1.50000001]
```

scipy.stats

```
from scipy.stats import norm

print(norm.mean())
print(norm.std())
print(norm.rvs(size=5))
```

scipy.linalg

```
from scipy import linalg
import numpy as np

A = np.array([[1, 2], [3, 4]])
linalg.inv(A)
```

```
array([[-2. ,  1. ],
       [ 1.5, -0.5]])
```

scikit-learn (Machine Learning in Python)

- Built on NumPy & SciPy
- Implements classical ML algorithms
- Consistent API design
  - *Step 1: Create a model object*
  - *Step 2: Fit the model*
  - *Step 3: Make predictions*
- Easy model training & evaluation
  - *Train/test splitting, Cross-validation, performance metric, etc.*

scikit-learn (Machine Learning in Python)

- **The scikit-learn workflow**
  - Prepare data
  - Split into train/test/validation
  - Initialize model
  - Fit model
  - Predict
  - Evaluate

Preparing Data

```
import pandas as pd

df = pd.read_csv("students.csv")

X = df[["math", "science", "english"]]
y = df["passed"]
```

Train/Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Training a Model

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)
```

Prediction & Evaluation

```
from sklearn.metrics import accuracy_scor
y_pred = model.predict(X_test) # prediction
print("Accuracy:", accuracy_score(y_test, y_pred)) # evaluation
```

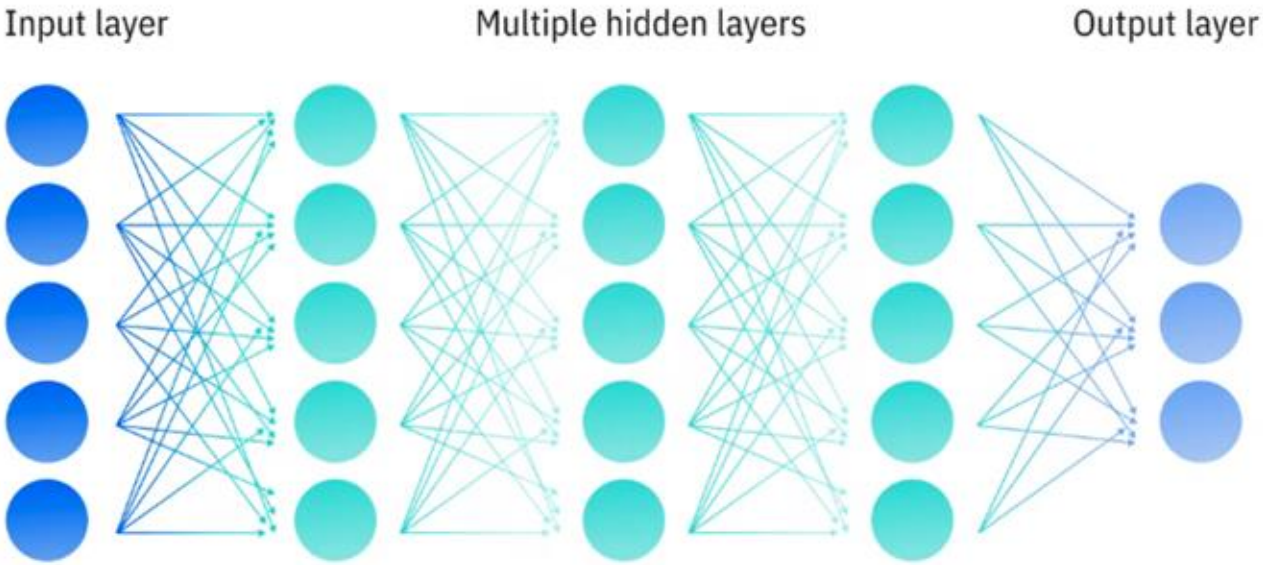
References & Resources

- Official Documentation
- Getting Started
- User Guide

Deep Learning

Deep Learning (Neural Networks with Many Layers)

- Deep learning uses multi-layered neural networks
- Learn from large datasets to make predictions automatically
- Deep learning dominates fields like: Computer Vision, Natural Language Processing, Large Language Models, Speech Recognition



A standard feedforward neural network with 3 hidden layers.

[What is Deep Learning?]

PyTorch (Deep Learning Framework)

PyTorch allows us to:

- Build neural networks (layers, loss functions, optimizers)
- Train them efficiently
- Use GPUs for acceleration

Neural Network Example

```
import torch.nn as nn

class SimpleNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(3, 5)
        self.output = nn.Linear(5, 1)

    def forward(self, x):
        x = torch.relu(self.hidden(x))
        x = torch.sigmoid(self.output(x))
        return x
```

Python for ML

- Using Pandas to clean and prepare data
- Using NumPy to handle numerical computation
- Using SciPy for scientific computing
- Using scikit-learn for classical machine learning
- Using Matplotlib and Seaborn to visualize data
- Using PyTorch to build and train neural networks